

ML basics & Optimization

Date: 2026-01-15

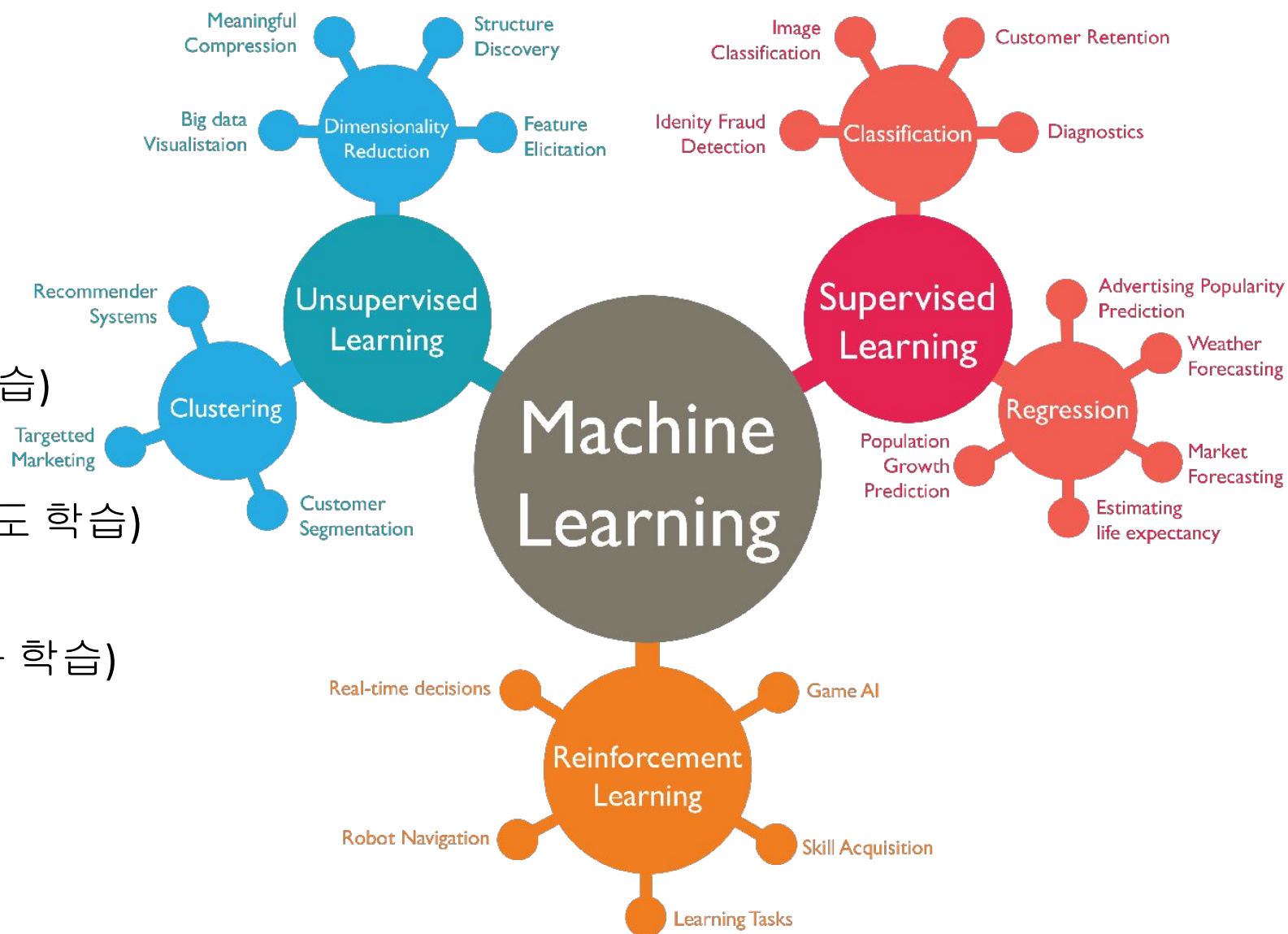
Presenter: Hyeonseong Sim

What is Learning?

-
- 데이터 $\mathcal{D}$ 로부터 함수(모델) f_{θ} 의 파라미터 θ 를 최적화(optimization)하는 과정
- 데이터 $\mathcal{D}$ 를 가장 잘 설명하는 수학적 함수를 찾아가는 과정
- 파라미터 θ 를 업데이트하는 과정

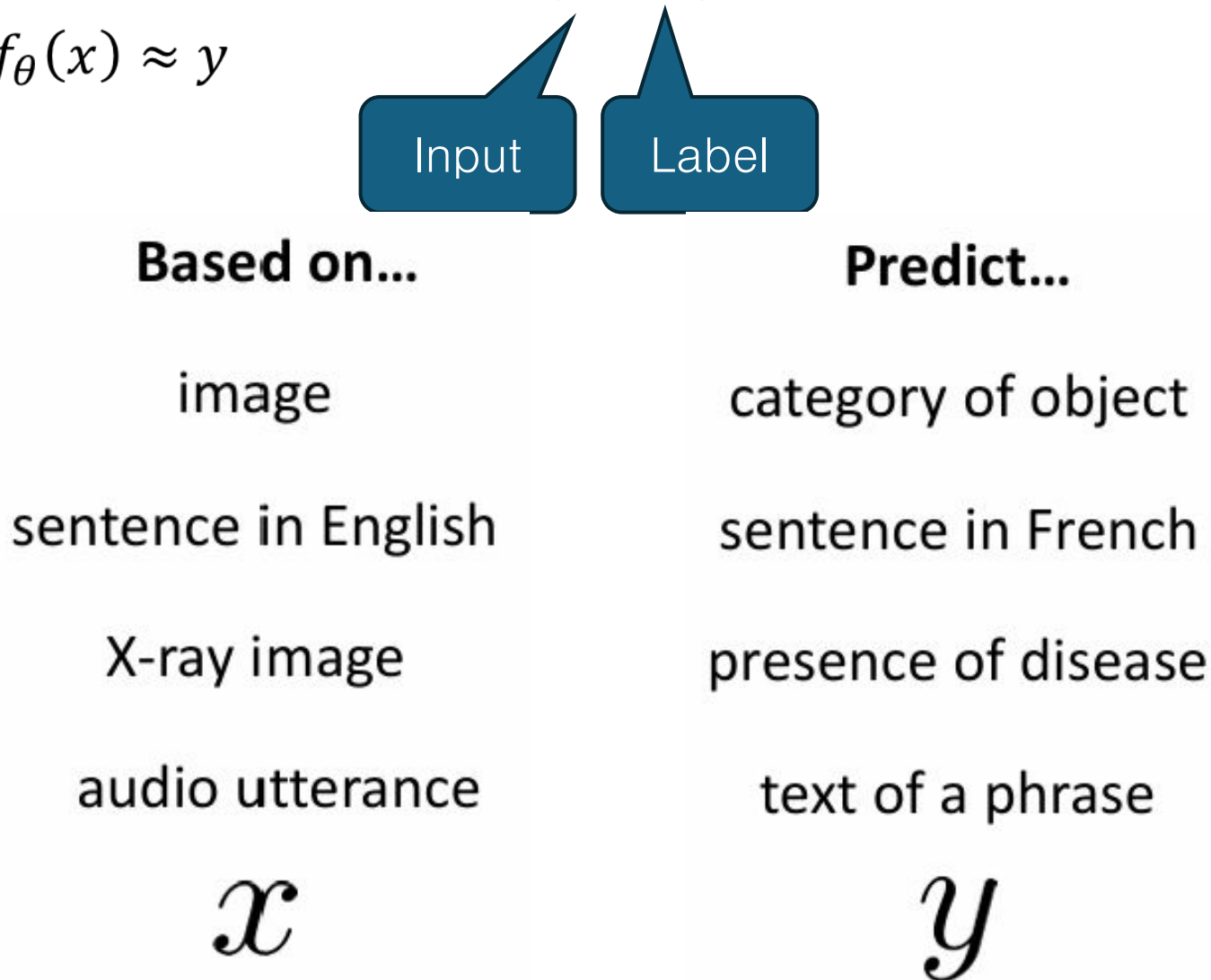
Different Types of Learning Problems

- Supervised Learning (지도 학습)
- Unsupervised Learning (비지도 학습)
- Reinforcement Learning (강화 학습)



Supervised Learning (지도 학습)

- Dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Goal: Learn $f_{\theta}(x) \approx y$



Supervised Learning (지도 학습)

- Dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Goal: Learn $f_{\theta}(x) \approx y$
- Examples: Regression (회귀), Classification (분류)

Supervised Learning (지도 학습) – Regression (회귀)

- Dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Goal: Learn $f_{\theta}(x) \approx y$
- Find a fitted function ($x \rightarrow y$)

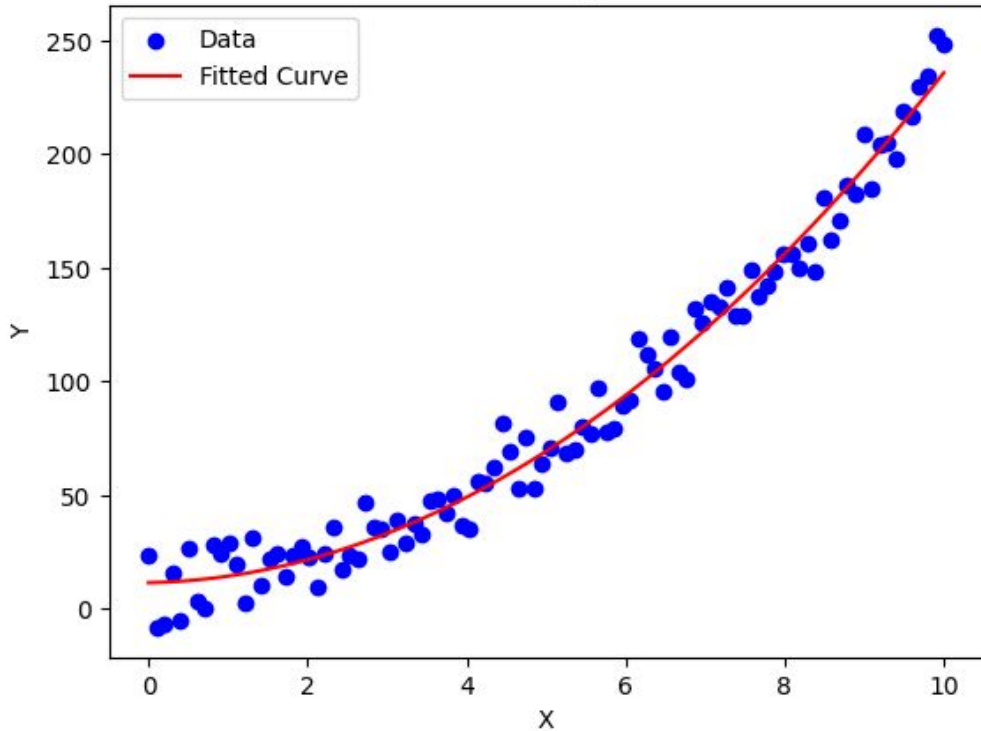
환자 ID	몸무게	키	흡연여부	기대수명
1	60 kg	5.5 ft	1	66
2	65 kg	5.0 ft	0	74
3	55 kg	6.0 ft	1	78



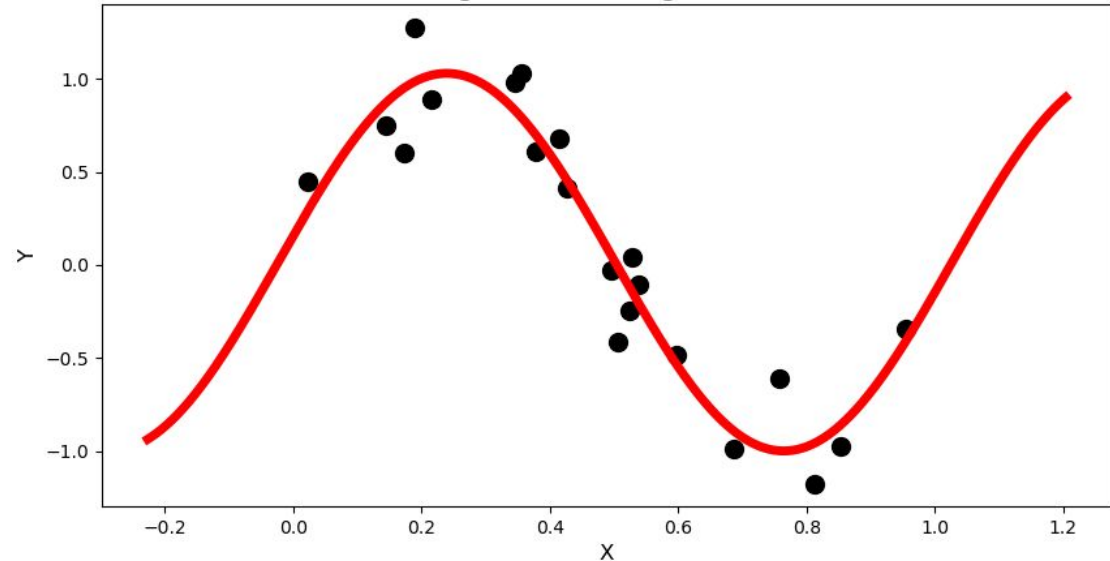
Supervised Learning (지도 학습) – Regression (회귀)

- Dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Goal: Learn $f_{\theta}(x) \approx y$
- Find a fitted function ($x \rightarrow y$) for continuous outputs

Nonlinear Regression



Non-Linear Regression Fitting - Gradient Descent



Supervised Learning (지도 학습) – Classification (분류)

- Dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Goal: Learn $f_{\theta}(x) \approx y$
- Find a decision boundary that separates classes



$y = \text{dog}$



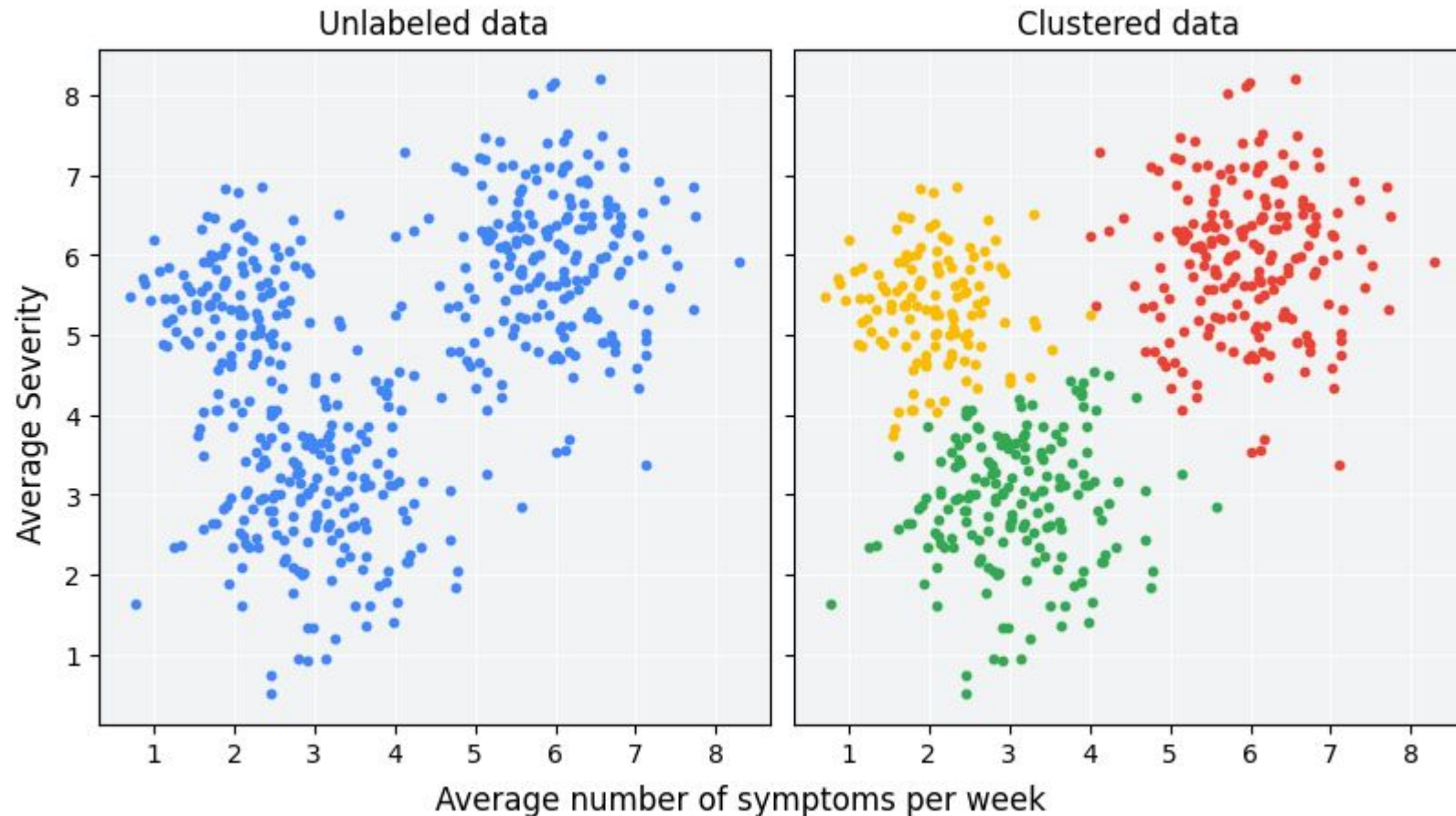
$y = 3$

Unsupervised Learning (비지도 학습)

- Dataset: $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ No labels y !
- Goal: Discover hidden structure in data
- Examples: Clustering, Dimensionality Reduction

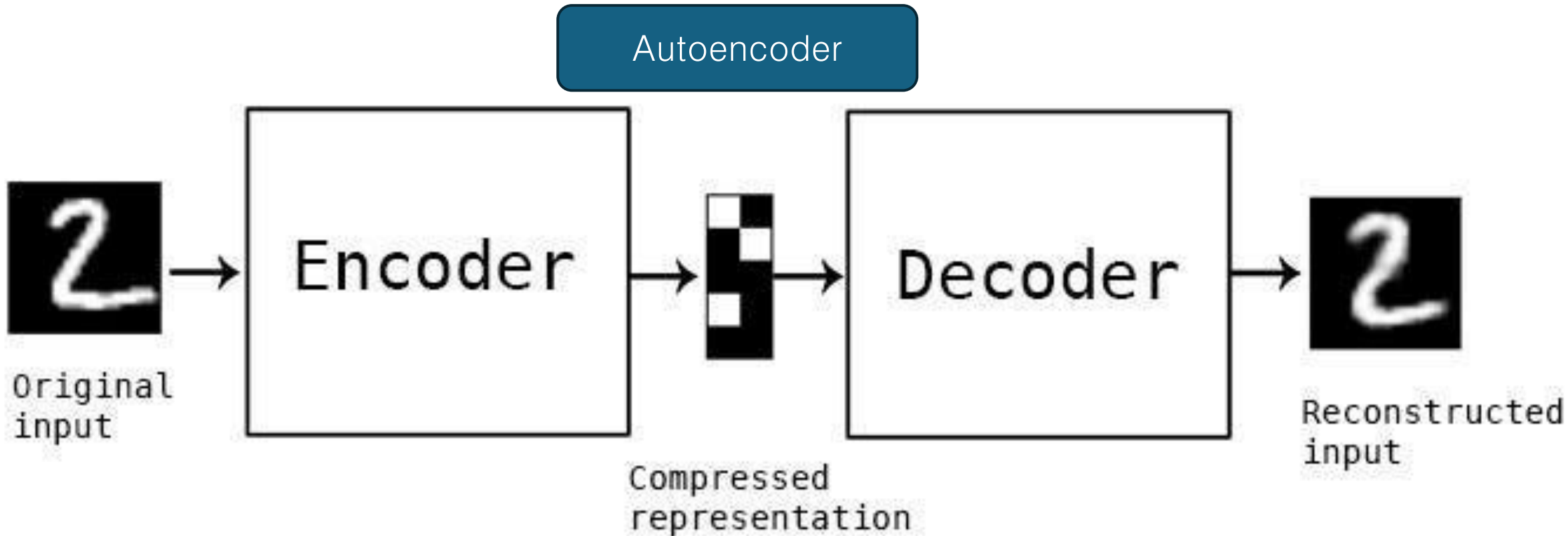
Unsupervised Learning (비지도 학습) – Clustering (군집화)

- Dataset: $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ No labels y !
- Goal: Discover hidden structure in data
- Find groups of similar data points



Unsupervised Learning (비지도 학습) – Dimensionality Reduction (차원 축소)

- Dataset: $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ No labels y !
- Goal: Discover hidden structure in data
- Find low-dimensional representations (e.g. Autoencoder)



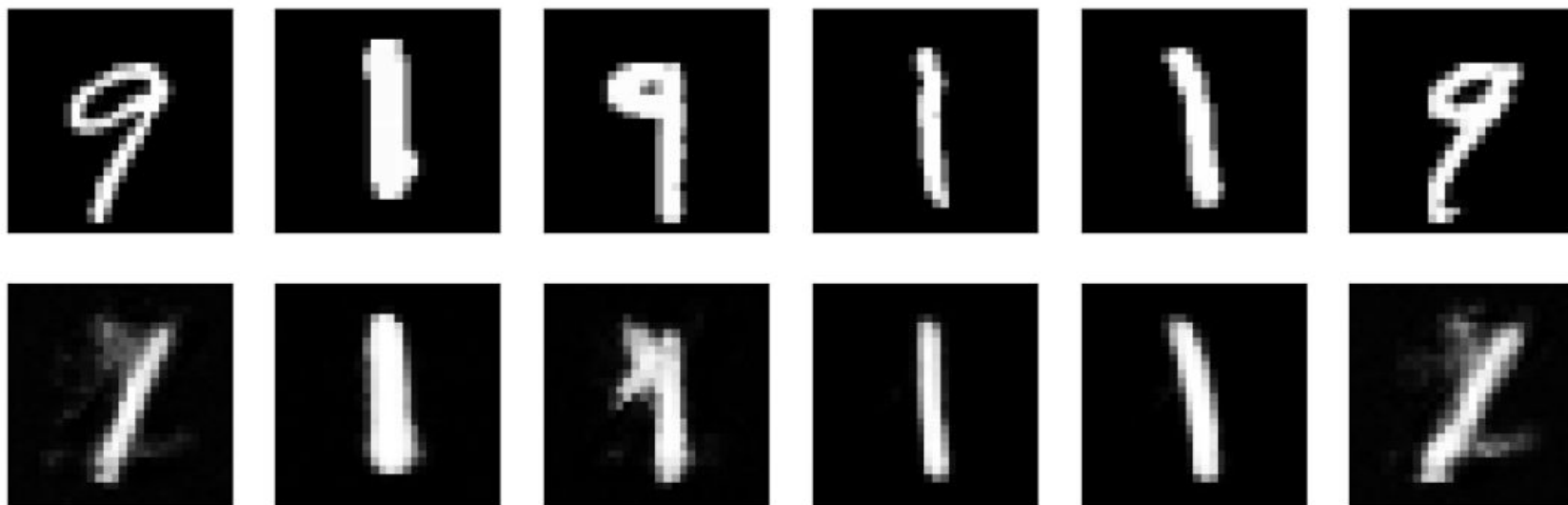
Unsupervised Learning (비지도 학습) – Dimensionality Reduction (차원 축소)

- Dataset: $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ No labels y !
- Goal: Discover hidden structure in data
- Find low-dimensional representations (e.g. Autoencoder)

- Application: Anomaly Detection (이상 탐지)

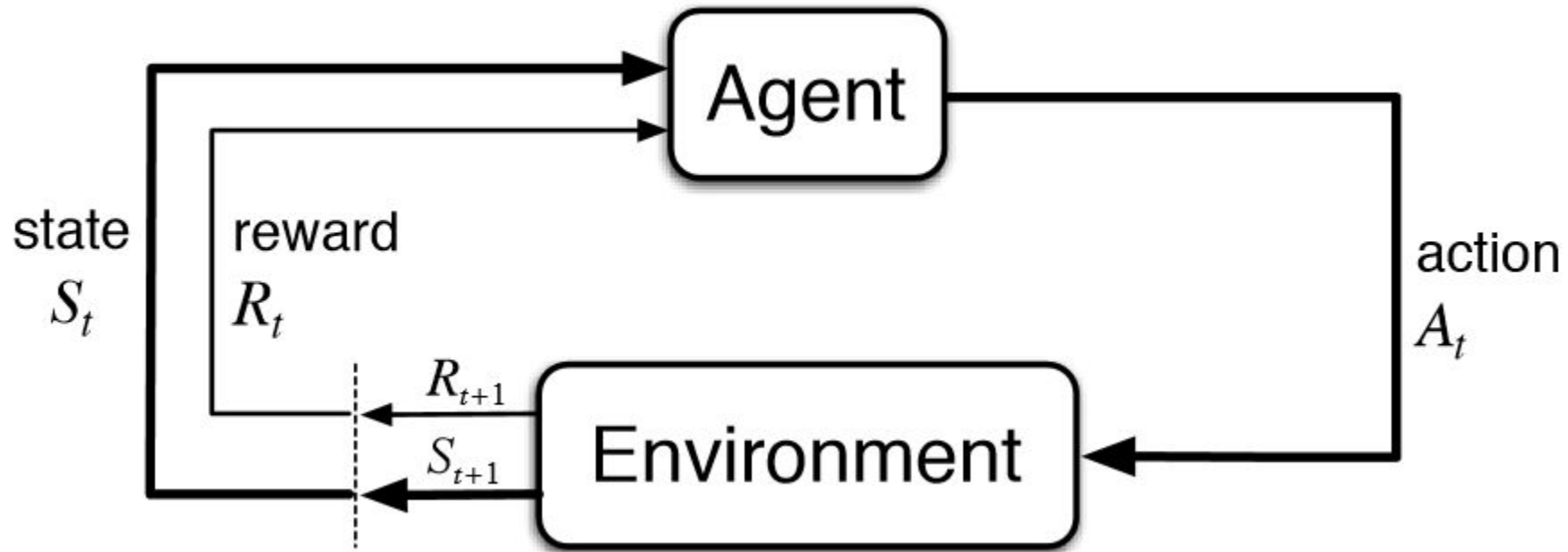
Autoencoder

1: Normal data
9: Anomaly data



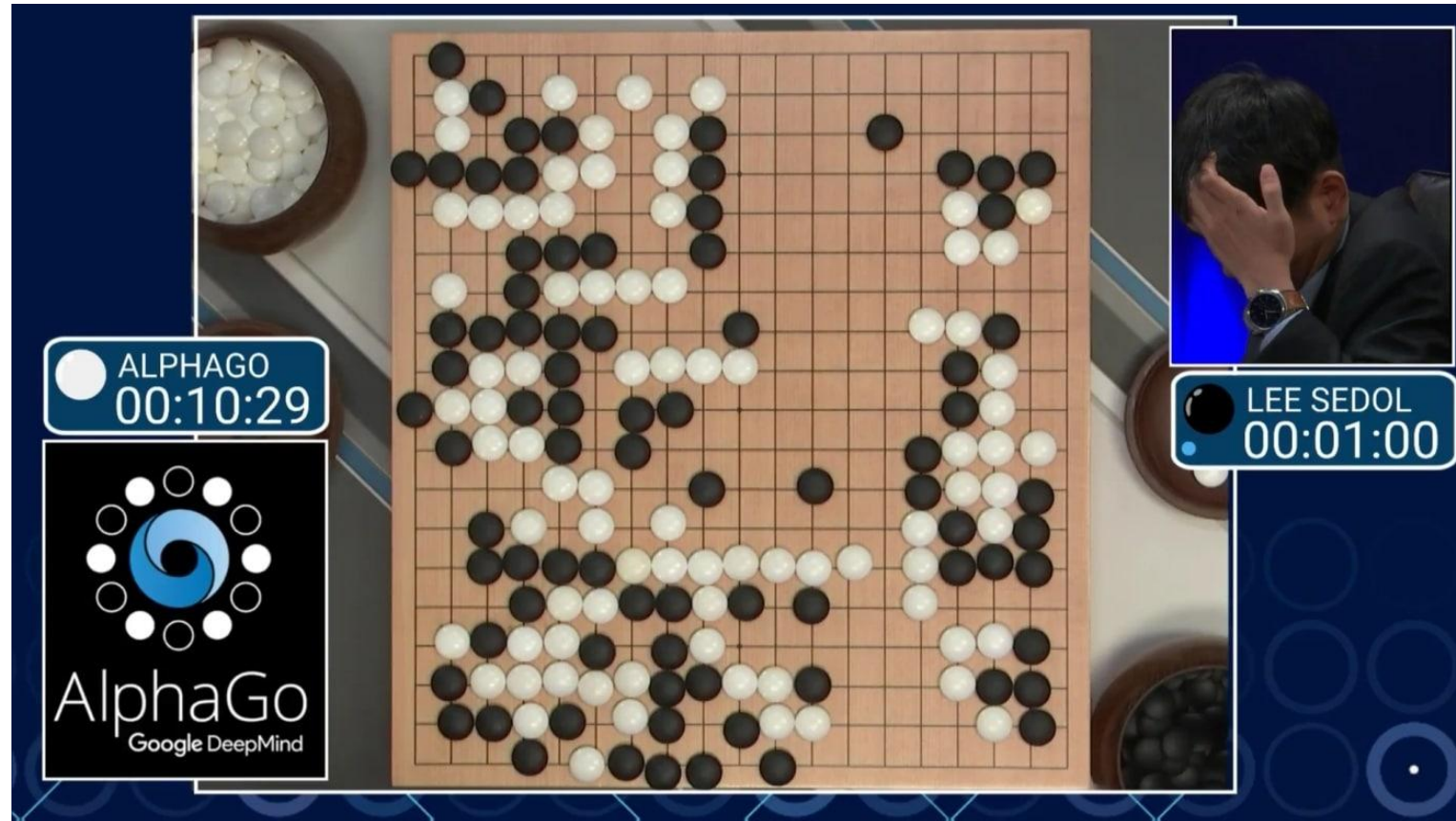
Reinforcement Learning (강화 학습)

- Agent가 Environment와 상호작용
- Goal: Maximize cumulative reward
- Find an optimal policy $\pi(a|s)$



Reinforcement Learning (강화 학습)

- Agent가 Environment와 상호작용
- Goal: Maximize cumulative reward
- Find an optimal policy $\pi(a|s)$
- Agent: AlphaGo
- State: 현재 바둑판 상태
- Action: 특정 위치에 바둑돌 놓기
- Reward
 - Win: +10
 - Lose: -10
 - Each move: -0.1



Supervised Learning

(지도 학습)

Loss Function

- Goal: **Learn** $f_{\theta}(x) \approx y$ $\Leftarrow$ 컴퓨터가 이해하지 못함
- Loss function: 학습시키고 있는 모델 f 가 잘하고 있는지, 못하고 있는지를 **숫자**로 표현
(모델의 실력을 점수로 매긴 것)
- 모델의 성능이 나쁠수록 loss 값 $\uparrow$, 좋을수록 loss 값 $\downarrow$
- Same Goal: **Minimize loss** $\mathcal{L}$

Prediction of Model

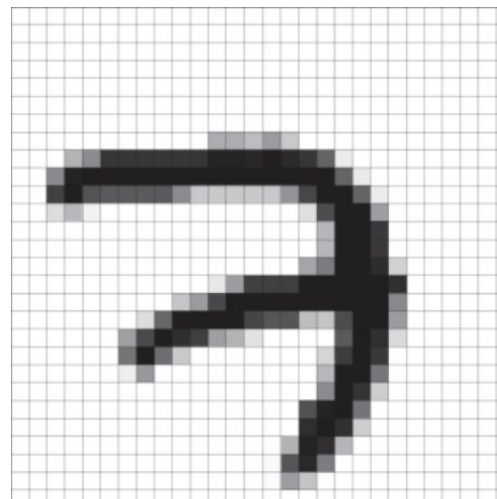
- Goal: Learn $f_{\theta}(x) \approx y$
- Goal: Minimize loss $\mathcal{L}$
- Predict exact labels
 - $f_{\theta}(x_i) = y_i$
- Predict probabilities
 - $f_{\theta}(x_i) = p(y|x_i)$

Predict Exact Labels

- If the answer is correct, $f_{\theta}(x_i) = y_i$, loss $\mathcal{L} = 0$.
- If the answer is wrong, $f_{\theta}(x_i) \neq y_i$, loss $\mathcal{L} = 1$.
- 모델 개선이 불가능함
- Uncertainty (불확실성)를 전혀 나타내지 못함



$$\begin{aligned}y &= \text{cat} \\ f_{\theta}(x) &= \text{cat} \\ \mathcal{L} &= 0\end{aligned}$$



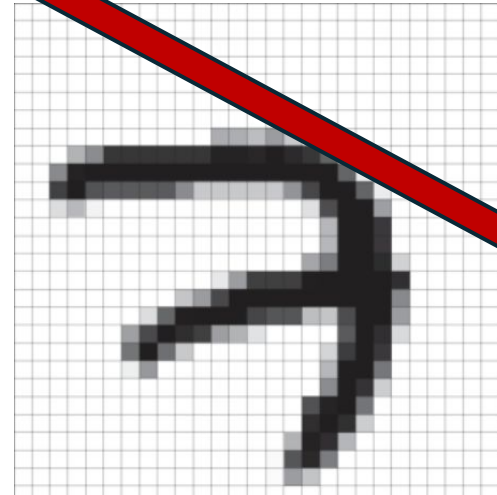
$$\begin{aligned}y &= 7 \\ f_{\theta}(x) &= 7 \\ \mathcal{L} &= 0\end{aligned}$$

Predict Exact Labels

- If the answer is correct, $f_{\theta}(x_i) = y_i$, loss $\mathcal{L} = 0$.
- If the answer is wrong, $f_{\theta}(x_i) \neq y_i$, loss $\mathcal{L} = 1$.
- 모델 개선이 불가능함
- Uncertainty (불확실성)를 전혀 나타내지 못함



$$\begin{aligned}y &= \text{cat} \\ f_{\theta}(x) &= \text{cat} \\ \mathcal{L} &= 0\end{aligned}$$



$$\begin{aligned}y &= 7 \\ f_{\theta}(x) &= 7 \\ \mathcal{L} &= 0\end{aligned}$$

Predict Probabilities

- 모델 개선 가능
- 모델 출력에 Uncertainty (불확실성) 내포



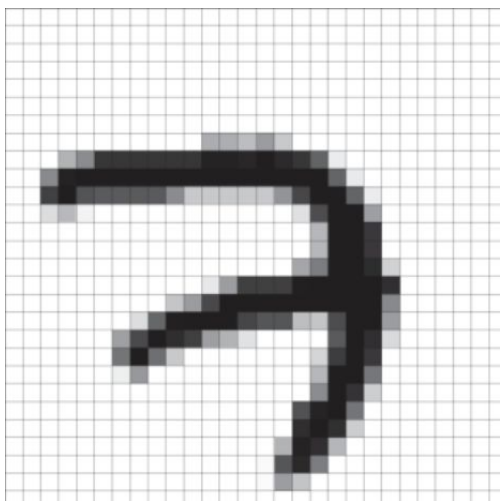
$y = \text{cat}$

$$f_{\theta}(x) = \begin{matrix} \text{dog} & \text{cat} \\ (40\%, 60\%) \end{matrix}$$

↓ 개선

$$f_{\theta}(x) = (30\%, 70\%)$$

Predict $p(y|x)$



$y = 7$

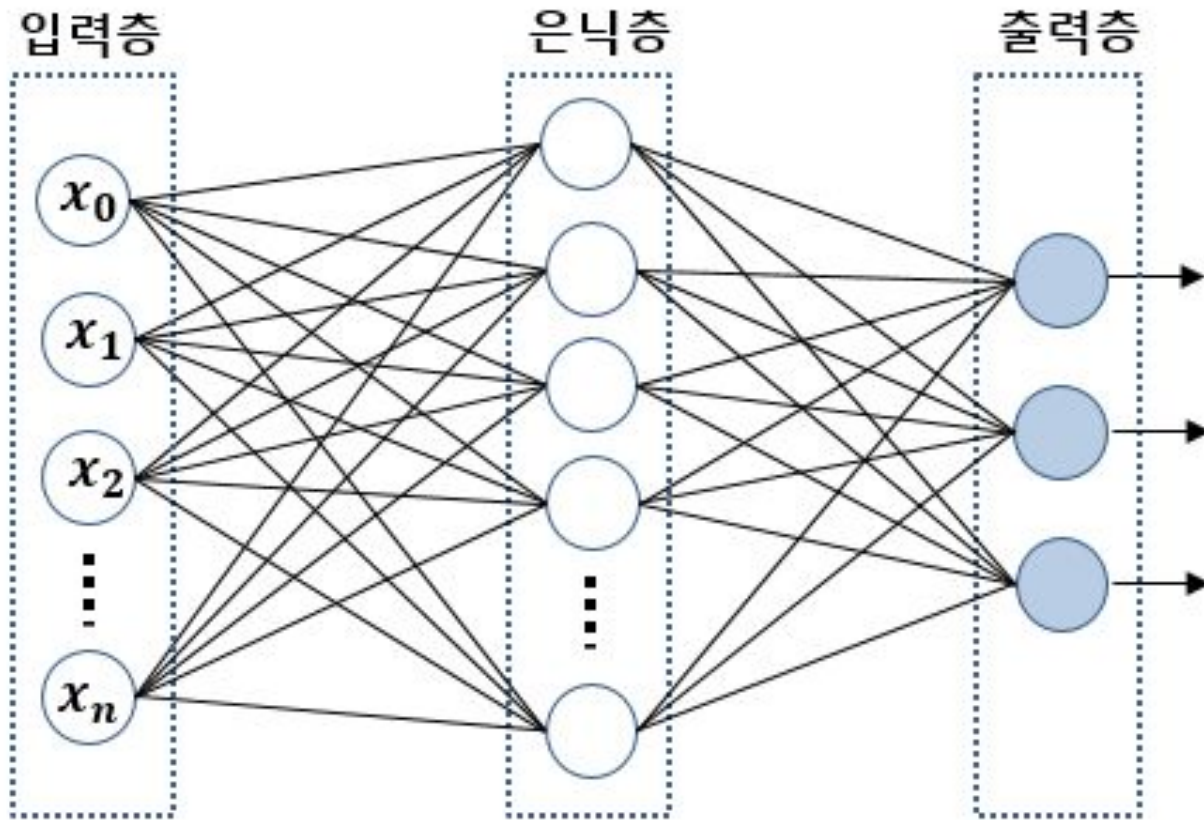
$$f_{\theta}(x) = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ (0, 0, 0, 40\%, 0, 0, 0, 50\%, 5\%, 5\%) \end{matrix}$$

↓ 개선

$$f_{\theta}(x) = (0, 0, 0, 35\%, 0, 0, 0, 60\%, 2\%, 3\%)$$

Softmax

- How to change output vector to probability



Output vector
(-1.1, 0, 0.3, 1.2, 0.4, -0.2,
-0.6, 0.5)

결과값

↓ Softmax

Output probability
(0.03, 0.1, 0.13, 0.31, 0.14, 0.08,
0.05, 0.16)

Softmax

- For all $i \in \{1, 2, \dots, N\}$, $0 \leq p(y = i|x) \leq 1$
- $\sum_{i=1}^N p(y = i|x) = 1$

N possible labels

$p(y|x)$ – vector with N elements

$f_{\theta}(x)$ – vector-valued function with N outputs

$$p(y = i|x) = \text{softmax}(f_{\theta}(x))[i] = \frac{\exp(f_{\theta,i}(x))}{\sum_{j=1}^N \exp(f_{\theta,j}(x))}$$

ML Method

1. Model class 정의
2. **Loss function (손실 함수) 정의**
3. **Optimizer 선택**
4. GPU를 사용해 학습

How is a Dataset Generated?: Probabilistic Perspective

- Data 생성에 대한 가정

- Data는 결정론적으로 만들어지지 않음.
- 각 data는 어떤 확률 분포에서 추출(sampling)된다고 가정.
- 입력 x 가 주어지면, 정답(label) y 는 확률적으로 결정됨.

$$\begin{aligned}x &\sim p(x) \\ y &\sim p(y|x) \\ \Rightarrow (x, y) &\sim p(x, y)\end{aligned}$$

- 우리는 이 분포 자체를 알 수 없음. 오직 sample들만 관측 가능.
- Training dataset: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- i.i.d. 가정: Independent and Identically Distributed
- 각 data 쌍들이 동일한 분포 $p(x, y)$ 에서 독립적으로 추출되었다고 가정.
 $(x_i, y_i) \sim p(x, y)$

Modeling Data Distribution

- Goal: x 와 y 사이의 조건부 분포 $p(y|x)$ 를 모델링
- 입력 x 가 주어졌을 때 정답(label) y 가 나타날 조건부 확률 $p(y|x)$ 를 예측
- 그러나 실제 분포를 알 수 없으므로 파라미터 θ 를 가진 모델 f_θ 로 이를 근사함.
- **Goal: $p(y|x)$ 를 가장 잘 흉내내는 $p_\theta(y|x)$ 찾기** $\leftarrow$ Learning

Maximum Likelihood Estimation (MLE)

- $p(\mathcal{D}) = \prod_i p(x_i)p(y_i|x_i)$: dataset $\mathcal{D}$ 가 존재할 확률
- Likelihood: 파라미터 θ 가 주어졌을 때, 관측된 data $\mathcal{D}$ 가 나타날 확률
- $p(\mathcal{D}|\theta) = \prod_i p(x_i)p_\theta(y_i|x_i)$
- 이미 dataset $\mathcal{D}$ 가 관측되었으므로(training dataset), $p(\mathcal{D}|\theta)$ 가 최대가 되도록 하는 파라미터 θ 를 찾아야 함.
- 최적의 파라미터 $\theta^* = \operatorname{argmax}_\theta \prod_i p(x_i)p_\theta(y_i|x_i)$: Likelihood를 최대로 만드는 θ
 $\Leftarrow$ Maximum likelihood estimation (MLE)

Negative Log-Likelihood (NLL)

- $p(\mathcal{D}|\theta) = \prod_i p(x_i)p_\theta(y_i|x_i)$
- 확률은 $0 \leq p(x_i), p_\theta(y_i|x_i) \leq 1$ 이므로 곱할수록 매우 작아짐.
- $\log p(\mathcal{D}|\theta) = \sum_i [\log p(x_i) + \log p_\theta(y_i|x_i)] = \sum_i \log p_\theta(y_i|x_i) + \text{constant}$
- $\therefore \theta^* = \operatorname{argmax}_\theta \sum_i \log p_\theta(y_i|x_i)$
- $\theta^* = \operatorname{argmin}_\theta - \sum_i \log p_\theta(y_i|x_i) \quad \Leftarrow \text{Negative log-likelihood (NLL)}$

Define Loss Function: Use NLL

- Loss function: 모델의 성능이 나쁠수록 loss 값 $\uparrow$, 좋을수록 loss 값 $\downarrow$
- Goal: Minimize loss $\mathcal{L}$
- $\theta^* = \operatorname{argmin}_{\theta} - \sum_i \log p_{\theta}(y_i|x_i) \quad \Leftarrow \text{Negative log-likelihood}$
- $\mathcal{L}(\theta) = - \sum_i \log p_{\theta}(y_i|x_i)$
- $\theta^* = \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$

Quiz 1: Likelihood and Learning Objective

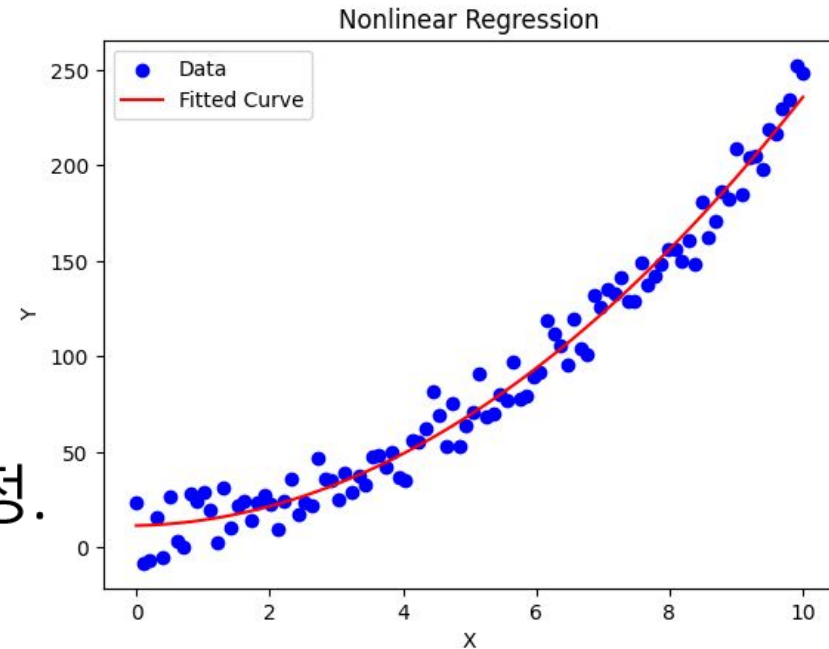
- Assume the following data generating process:
 - Training data consists of pairs $(x_1, y_1), \dots, (x_n, y_n)$
 - Each data point is sampled independently from an unknown true distribution.
 - We model the conditional distribution as $p_\theta(y|x)$
-
- Questions
 1. Write the likelihood of the training dataset under the model.
 2. Explain in words what Maximum Likelihood Estimation (MLE) is trying to do.
 3. Why do we minimize negative log-likelihood instead of maximizing likelihood directly?

Quiz 1 – Solution

1. Likelihood of the dataset: $\prod_{i=1}^n p_{\theta}(y_i|x_i)$
2. Interpretation of MLE:
 1. MLE chooses parameters θ that make the observed training data most probable under the assumed model.
 2. In other words, the model is adjusted so that the data we actually observed is likely to be generated by $p_{\theta}(y|x)$.
3. Why negative log-likelihood?
 1. The product of probabilities is numerically unstable and difficult to optimize.
 2. Taking the logarithm converts products into sums.
 3. Minimizing negative log-likelihood is mathematically equivalent to maximizing likelihood and is easier to optimize.

Various Loss Functions – Regression

- ϵ : 정답 y 와 모델 출력값 $f_{\theta}(x)$ 와의 오차
- $\epsilon = y - f_{\theta}(x) \Rightarrow y = f_{\theta}(x) + \epsilon$
- 오차 ϵ 이 평균이 0인 Gaussian Distribution을 따른다고 가정.
- $\epsilon \sim \mathcal{N}(0, \Sigma) \Rightarrow y|x \sim \mathcal{N}(f_{\theta}(x), \Sigma)$
- $$p_{\theta}(y|x) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp \left(-\frac{1}{2} (y - f_{\theta}(x))^{\top} \Sigma^{-1} (y - f_{\theta}(x)) \right)$$
- NLL 적용: $-\log p_{\theta}(y|x) = \frac{1}{2} \log(2\pi)^d |\Sigma| + \frac{1}{2} (y - f_{\theta}(x))^{\top} \Sigma^{-1} (y - f_{\theta}(x))$
- If $\Sigma = \sigma^2 I$, $-\log p_{\theta}(y|x) = \text{constant} + \frac{1}{2\sigma^2} \|y - f_{\theta}(x)\|^2$



Various Loss Functions – Regression

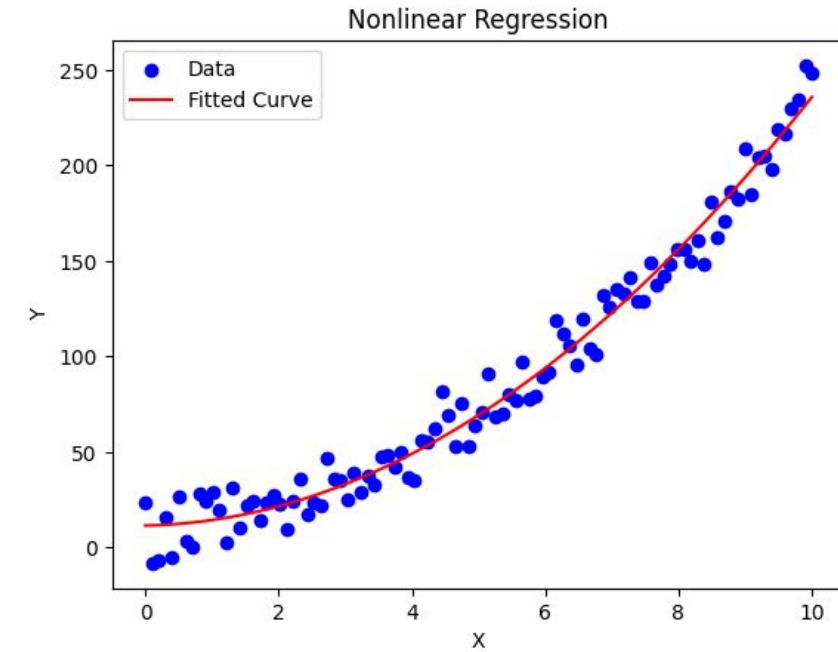
- If $\Sigma = \sigma^2 I$, $-\log p_{\theta}(y|x) = \text{constant} + \frac{1}{2\sigma^2} \|y - f_{\theta}(x)\|^2$

- Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$

- $\theta^* = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n -\log p_{\theta}(y_i|x_i)$

- If $\Sigma = \sigma^2 I$, $\theta^* = \operatorname{argmin}_{\theta} \frac{1}{2} \sum_{i=1}^n \|y - f_{\theta}(x)\|^2$

- Then, let $\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^n \|y - f_{\theta}(x)\|^2 \quad \Leftarrow \text{MSE loss (Mean Squared Error)}$

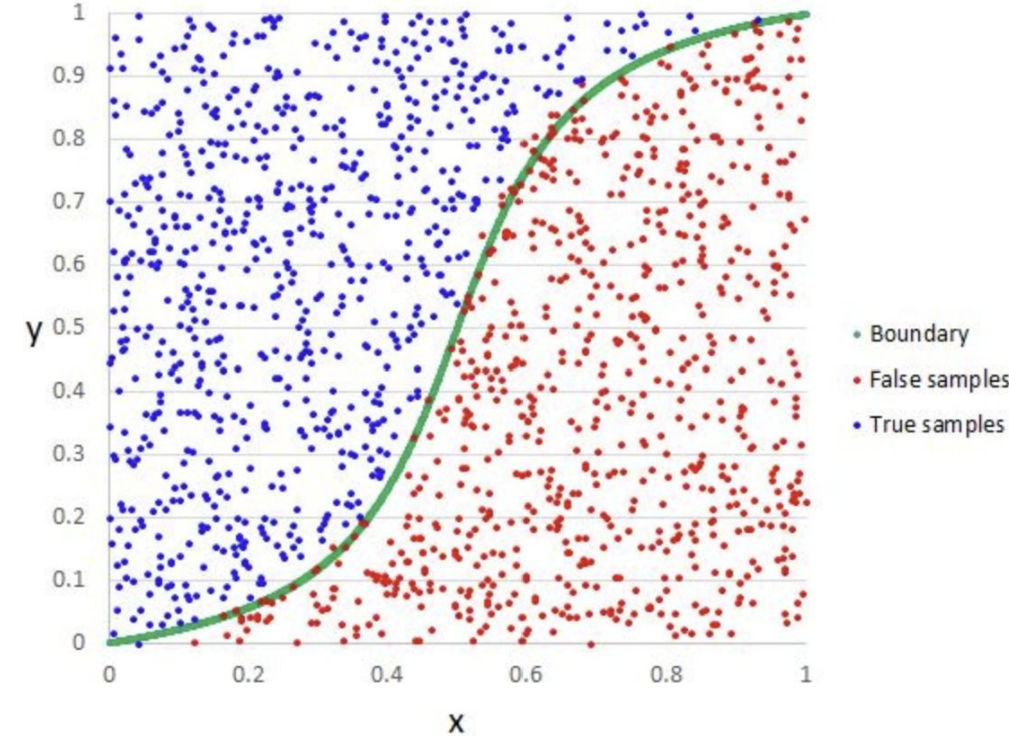


Various Loss Functions – Classification

- Label $y \in \{1, 2, \dots, K\}$: K 개의 class
- One-hot vector $t \in \{0, 1\}^K$
- $t_k = \begin{cases} 1, & y = k \\ 0, & y \neq k \end{cases}$

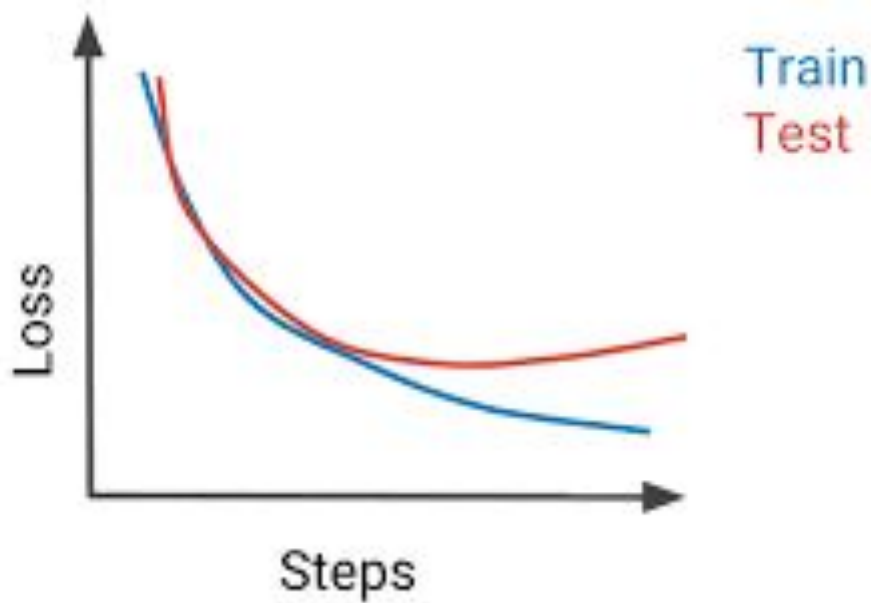
- 모델 출력값 $z = f_\theta(x) \in \mathbb{R}^K$
- Softmax: $p_k = p_\theta(y = k|x) = \frac{\exp z_k}{\exp z_1 + \exp z_2 + \dots + \exp z_K}$

- $\mathcal{L}(\theta) = -\log p_\theta(y|x) = -\log p_y \quad \Leftarrow \text{Cross-entropy loss}$



Bias, Variance, and Regularization

- Optimization으로 training loss 최소화 $\neq$ Test 성능이 좋음 (Generalization)



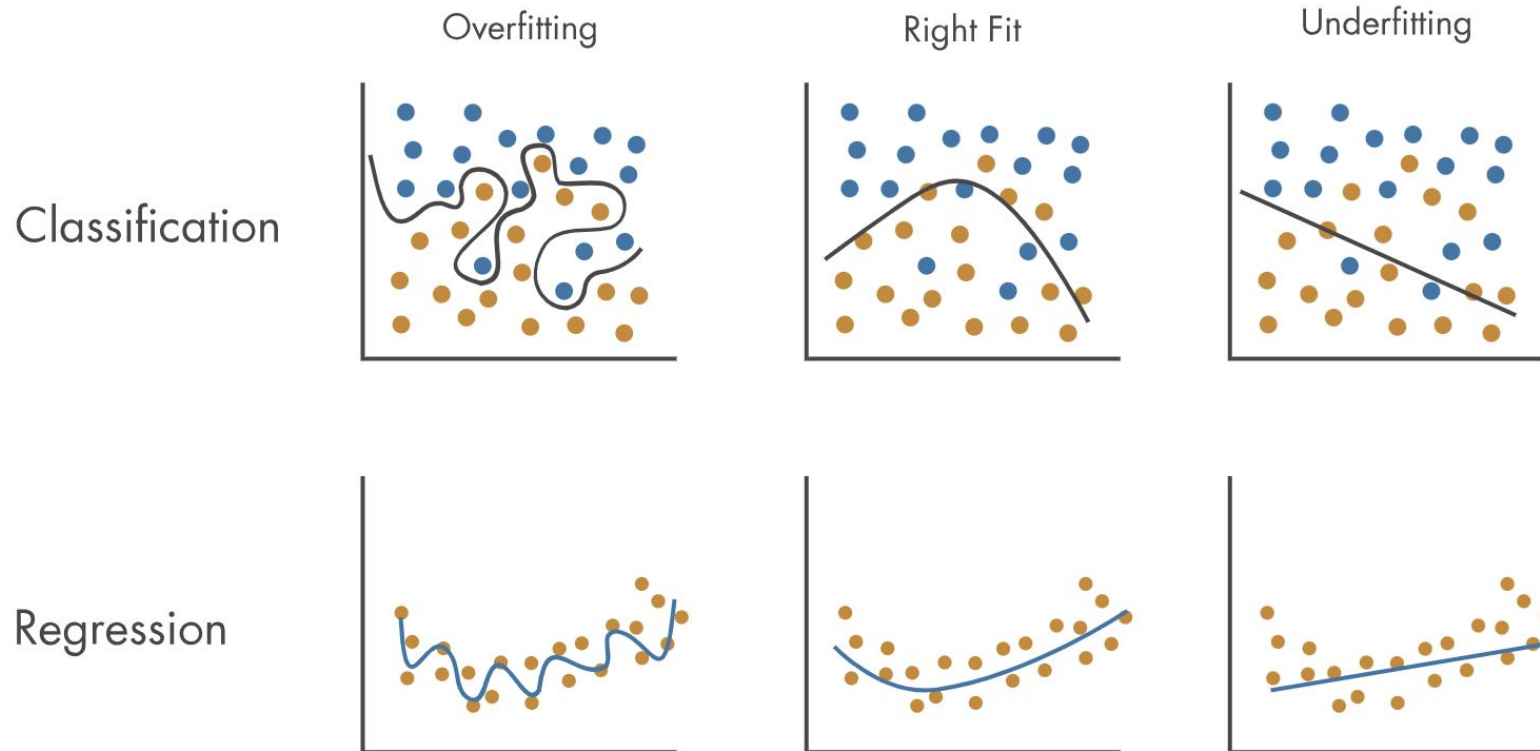
- 왜 training set에는 잘 맞는데, test set에는 잘 안 맞을까?

Empirical Risk vs. True Risk

- True risk: 실제 data 분포에서의 expected loss. 실제 data 분포 $p(x)$ 에서의 기대 오류.
- $\mathbb{E}_{x \sim p(x), y \sim p(y|x)} [\mathcal{L}(x, y, \theta)]$
- $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Empirical risk: 우리가 가진 dataset $\mathcal{D}$ 에서의 평균 loss. Dataset에서의 평균 오류.
- $\frac{1}{n} \sum_{i=1}^n \mathcal{L}(x_i, y_i, \theta)$
- 실제 줄이고 싶은 건 true risk이나 $p(x)$ 를 몰라 불가능하므로 empirical risk를 줄임으로써 근사함.

Overfitting vs. Underfitting

- Overfitting: Empirical risk low & True risk high
- Dataset이 너무 작거나 모델의 capacity(파라미터 수)가 너무 클 때 발생
- Underfitting: Empirical risk와 true risk 모두 high
- 모델의 capacity가 너무 작거나 optimizer 설정(learning rate 등)이 잘못되었을 때 발생



Bias-Variance Decomposition: Regression Analysis

- MSE Loss $\mathcal{L}(\theta) = \frac{1}{2} \|f_{\theta}(x) - y\|^2$
- x 에 대한 이상적인 정답 함수 $f(x)$, 관측된 label $y = f(x) + \epsilon$
- x 에 대한 모델 예측값 $f_{\theta}(x)$
- Let $\bar{f}(x) = \mathbb{E}_{\mathcal{D} \sim p(\mathcal{D})}[f_{\theta}(x)]$
- Error = $\mathbb{E}_{\mathcal{D} \sim p(\mathcal{D})}[\|f_{\theta}(x) - y\|^2] = \mathbb{E}_{\mathcal{D} \sim p(\mathcal{D})}[\|(f_{\theta} - \bar{f})(x) + (\bar{f}(x) - y)\|^2] =$
 $\underbrace{\mathbb{E}_{\mathcal{D} \sim p(\mathcal{D})}[\|f_{\theta}(x) - \bar{f}(x)\|^2]}_{\text{Variance}} + \underbrace{\|\bar{f}(x) - f(x)\|^2}_{\text{Bias}^2} + \text{Noise}$

Bias-Variance Tradeoff

- Variance 항: Dataset $\mathcal{D}$ 가 바뀔 때마다 예측이 얼마나 흔들리나?
- Dataset sampling에 민감한 정도.
- Bias² 항: 평균 예측이 ideal true function에서 얼마나 벗어나 있나?
- 모델이 충분한 표현력이 없어 생기는 성능 부족.
- If variance is too high → Overfitting
If bias is too high → Underfitting
- How to regulate bias-variance tradeoff: Regularization

Regularization

- 모델의 capacity를 직접 줄이는 대신, loss function에 penalty를 주어 Variance를 감소
→ Overfitting 억제
- L1 regularization: $\mathcal{L}(\theta) + \lambda \|\theta\|$
- 중요하지 않은 가중치를 0으로 만듦.
- L2 regularization: $\mathcal{L}(\theta) + \lambda \|\theta\|^2$
- 모든 가중치를 작게 유지.

Training vs. Validation vs. Test Sets

- Split training set → Training / Validation sets
- 각 hyperparameter (λ , α 등) 들마다 training set에 대하여 θ 를 훈련한 뒤 validation set으로 error 계산
- Best hyperparameters 선택 (lowest validation error)
- Test set으로 최종 성능 계산

- Overfitting: $\mathcal{L}_{val}(\theta) \gg \mathcal{L}_{train}(\theta)$
- Underfitting: $\mathcal{L}_{train}(\theta)$ not low enough

Quiz 2: Empirical Risk and Generalization

1. Why can we not compute the true risk directly?
2. What quantity do we minimize in practice instead?
3. Explain how model complexity affects bias and variance.

Quiz 2 – Solution

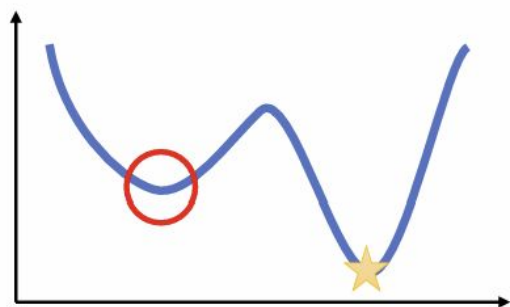
1. True risk cannot be computed because the true data distribution is unknown.
2. In practice, we minimize empirical risk, which is the average loss over the training dataset.
3. Effect of model complexity:
 1. Simple models:
 1. High bias
 2. Low variance
 2. Complex models:
 1. Low bias
 2. High variance

Increasing model complexity reduces bias but increases variance.

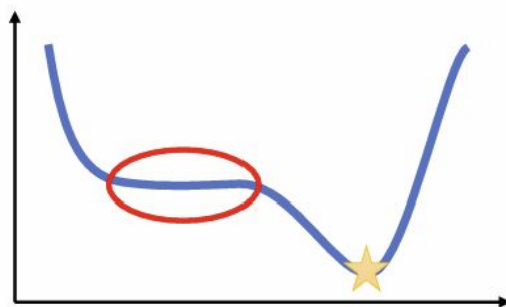
Regularization is used to control this tradeoff and improve generalization.

Loss Landscape

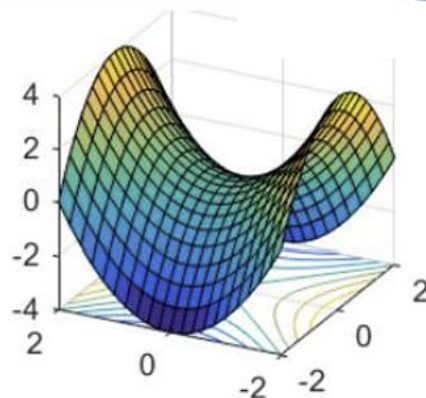
- $\mathcal{L}(\theta) = -\sum_i \log p_{\theta}(y_i|x_i)$: Scalar
- Visualization을 위해 $\theta \in \mathbb{R}^2$ 라 가정
- Goal: Find global(local) minima of $\mathcal{L}(\theta)$



the local optimum

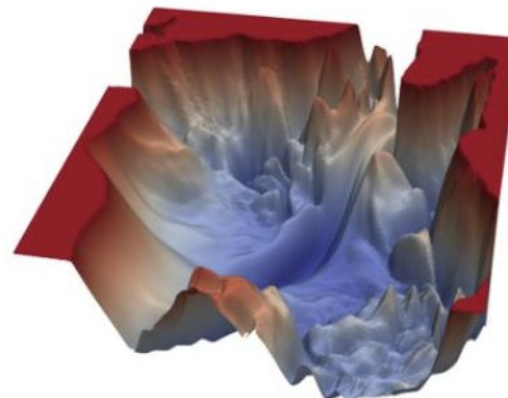


the plateau

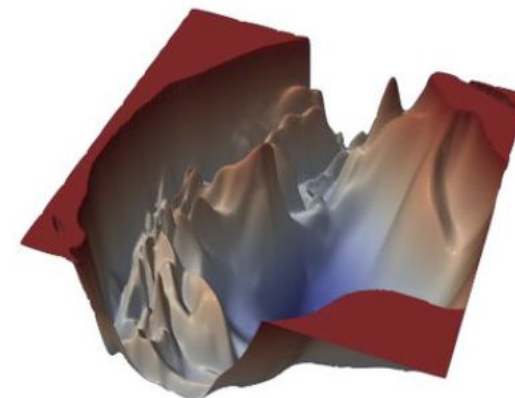


the saddle point

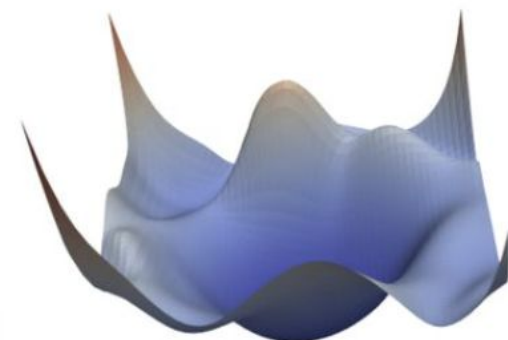
VGG-56



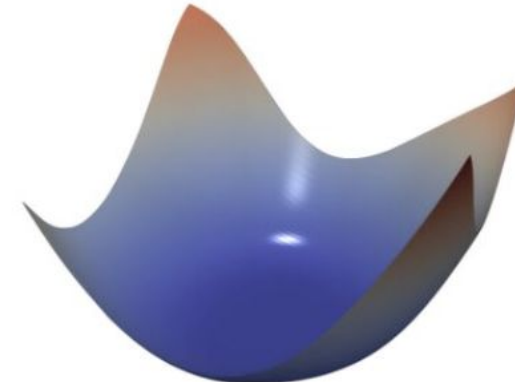
VGG-110



Resnet-56



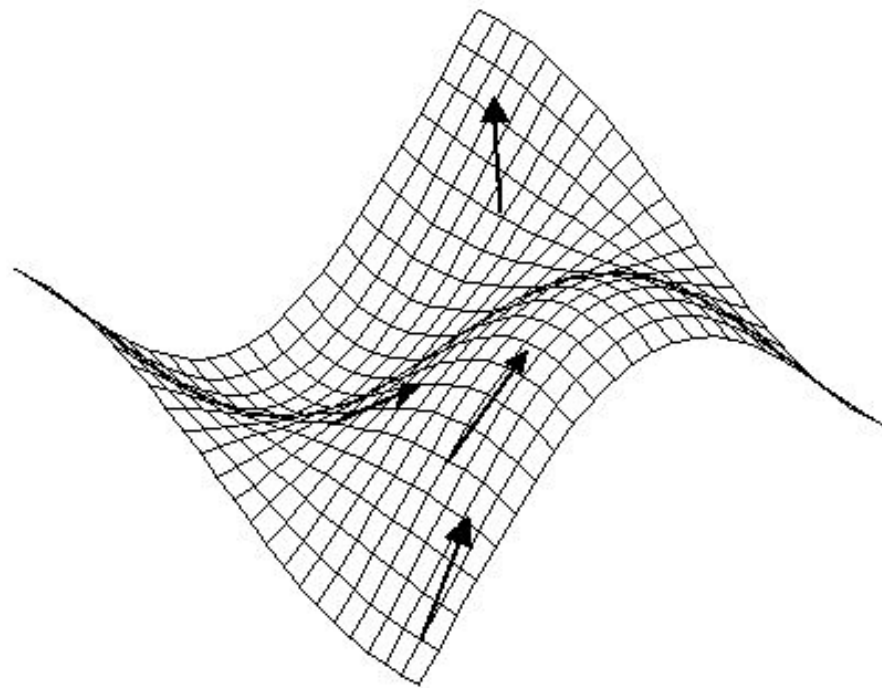
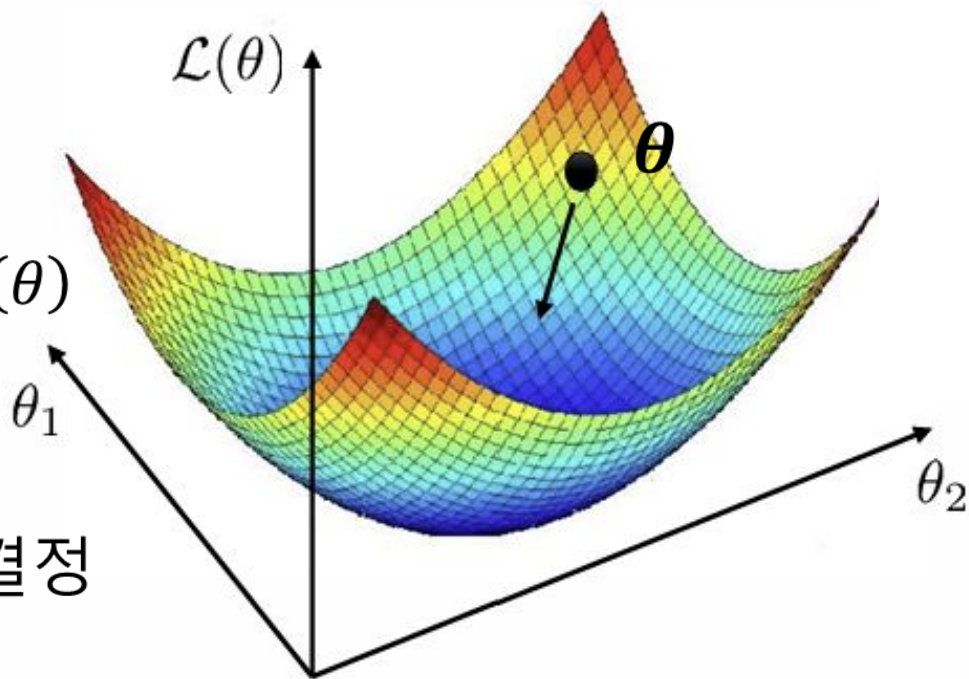
Densenet-121



Gradient Descent

- Gradient 방향: Local maxima를 향함
- Global(local) minima를 찾기 위해서는
- 현재 θ 에서 gradient 반대 방향으로 이동해야 함

- Optimization
- $\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$
- α : 보폭 결정
- $\nabla_{\theta} \mathcal{L}(\theta)$: 방향 결정

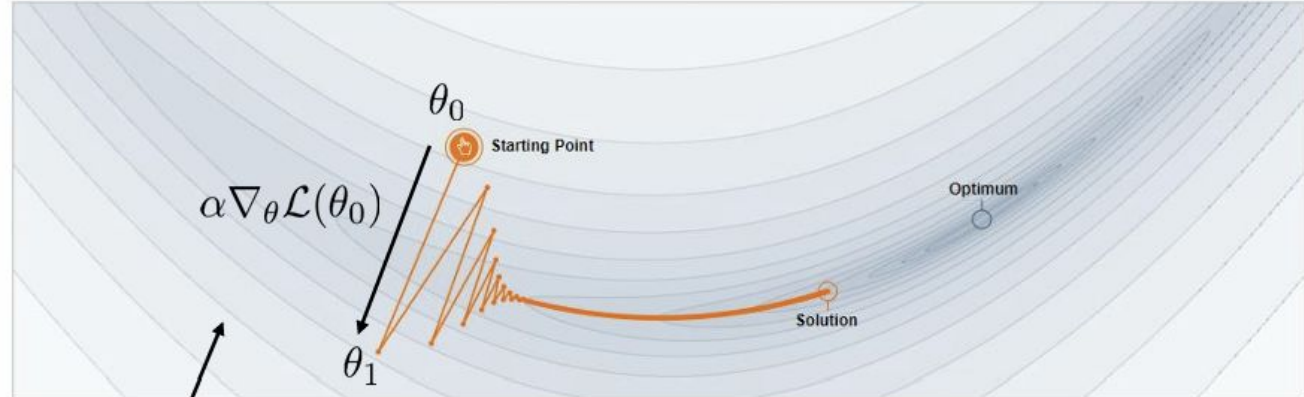
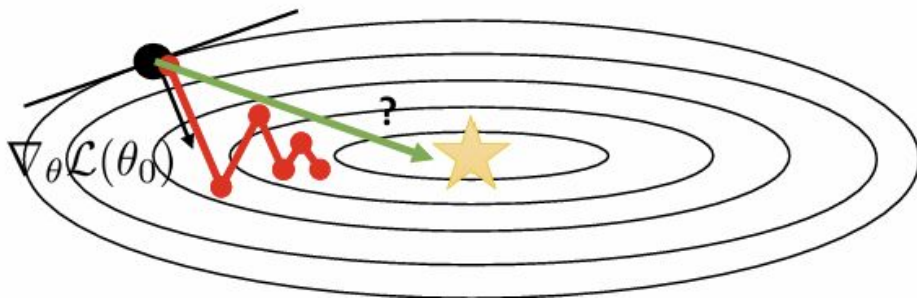


Gradient Vectors Shown at Several Points on the Surface of $\cos(x)\sin(y)$

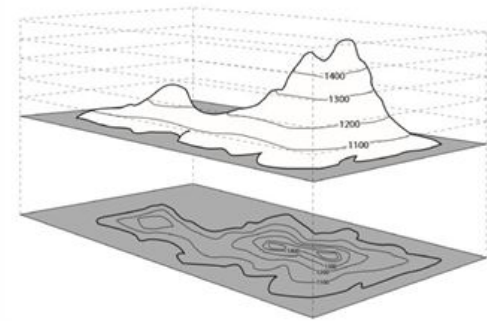
Gradient Descent



we don't always move toward the optimum!



level set contours
for all θ values along a line
 $\mathcal{L}(\theta)$ takes on the same value

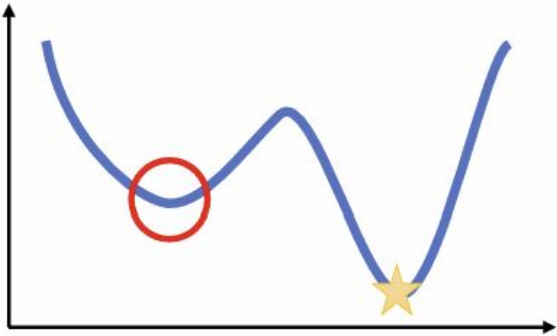


Stochastic Gradient Descent (SGD)

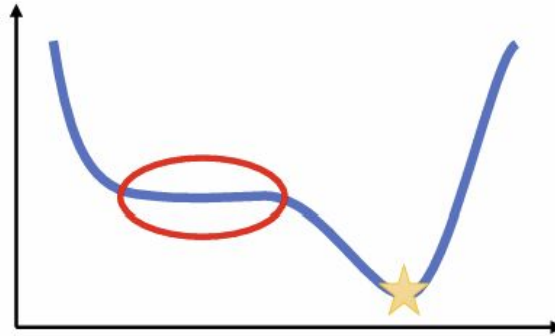
- Batch: 모델 파라미터 update를 위해 사용되는 training data sample의 묶음
- Batch size: 한 iteration당 모델에 전달되는 data sample 수 (B)
- Iteration: 한 batch를 사용해 파라미터를 update하는 과정
- Epoch: 전체 training dataset이 모델을 통과하여 update된 상태
- 전체 data 수가 N 일 때, $1 \text{ epoch} = \frac{N}{B} \text{ iteration}$
- $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}(\theta)$ where $\mathcal{L}(\theta) = -\frac{1}{B} \sum_{i=1}^B \log p_{\theta}(y_i|x_i)$
- Full gradient descent에 비해 noisy해 local minima 탈출에 도움을 줌

Learning Rate α

- Too small: 학습 느림 & local minima에 갇힐 위험

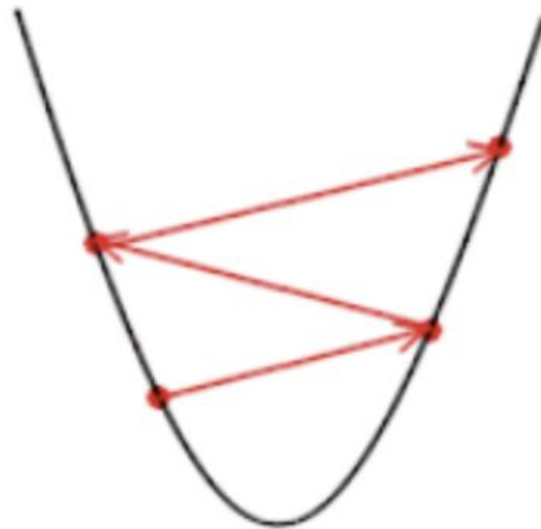
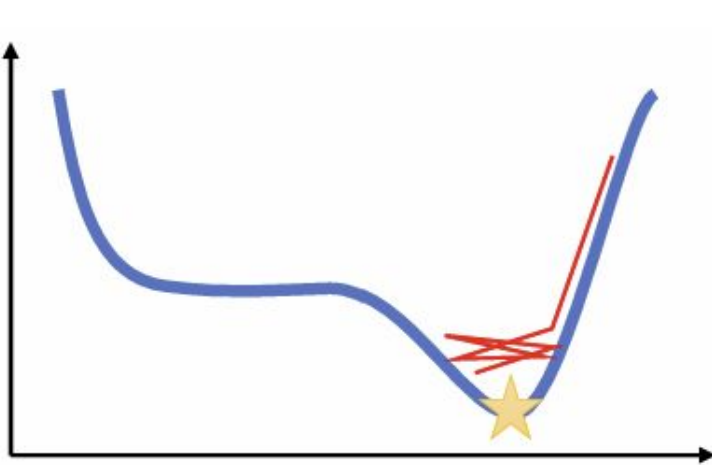


the local optimum



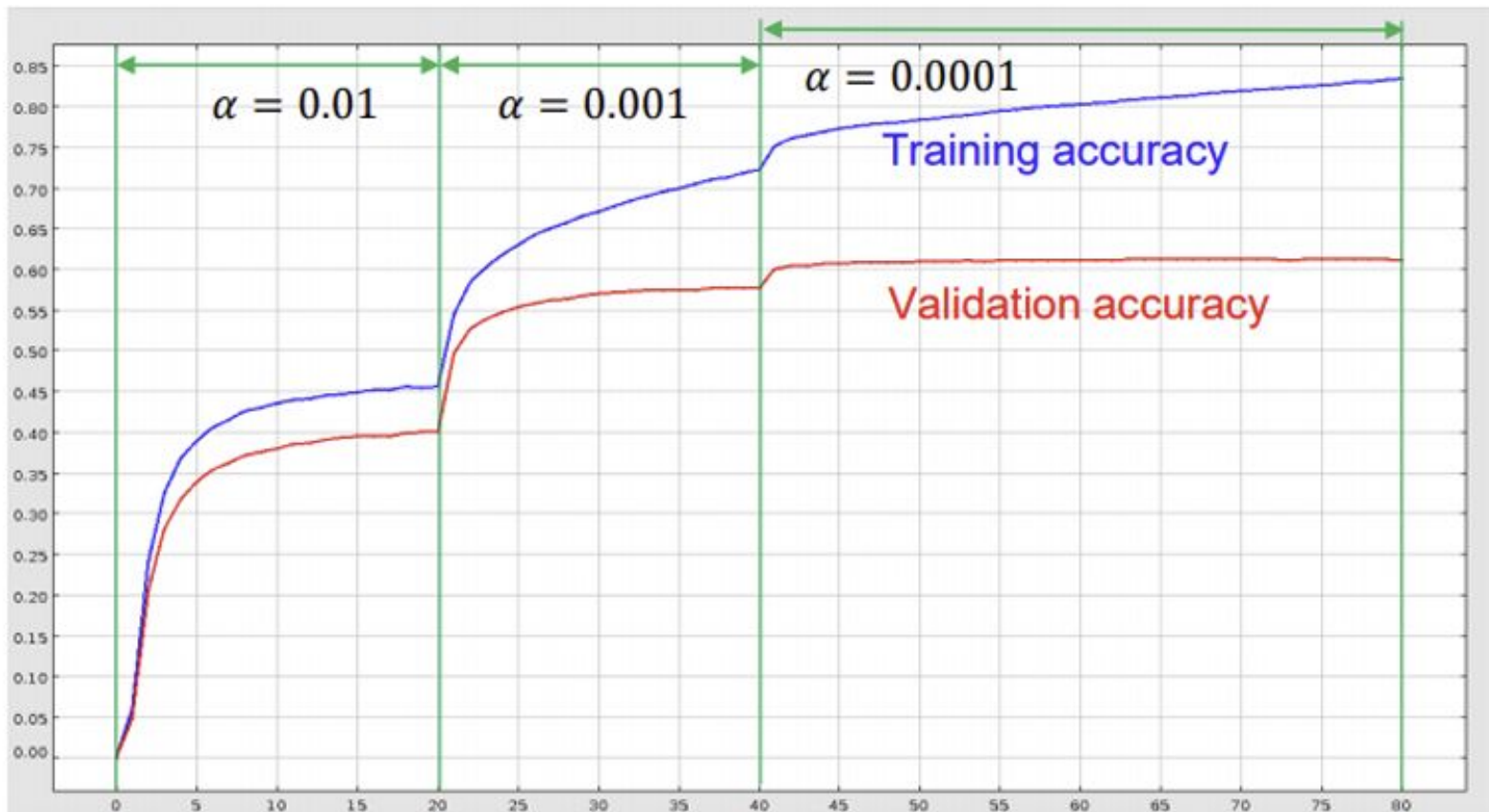
the plateau

- Too large: global minimum을 지나쳐 발산하거나 진동



Learning Rate Scheduling

- Step decay: 특정 epoch마다 α 감소 (예) 0.1배)
- Exponential decay: 학습 초기부터 빠르고 부드럽게 α 감소
- Cosine annealing: 코사인 곡선을 따라 학습 초기와 후기에는 점진적으로, 중기에는 빠르게 α 감소



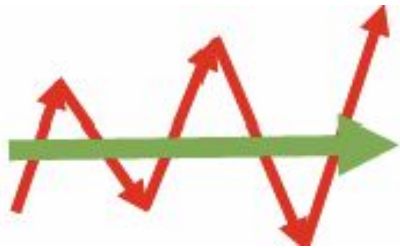
Various Optimizers

- Momentum
- AdaGrad
- RMSProp
- Adam

Various Optimizers – Momentum

- $v_{t+1} = \mu v_t - \alpha \nabla_{\theta} \mathcal{L}(\theta_t)$
- $\theta_{t+1} = \theta_t + v_{t+1}$
- 가던 방향으로 계속 가려는 관성을 줌
- Local minima나 Saddle point에서 멈추지 않고 탈출함

Gradient Descent



Momentum



Various Optimizers – AdaGrad

- $s_{t+1} = s_t + (\nabla_{\theta} \mathcal{L}(\theta_t))^2$
- $\theta_{t+1} = \theta_t - \alpha \frac{\nabla_{\theta} \mathcal{L}(\theta_t)}{\sqrt{s_{t+1} + \epsilon}}$
- 각 파라미터별로 학습률 다르게 설정 → 자주 변하는 파라미터는 조심스럽게, 드물게 변하는 파라미터는 과감하게 update
- Limitation: 학습이 진행될수록 s_t 가 계속 커져서 학습률이 0에 수렴

Various Optimizers – RMSProp

- $s_{t+1} = \rho s_t + (1 - \rho)(\nabla_{\theta} \mathcal{L}(\theta_t))^2$
- $\theta_{t+1} = \theta_t - \alpha \frac{\nabla_{\theta} \mathcal{L}(\theta_t)}{\sqrt{s_{t+1} + \epsilon}}$
- 최근 기울기에 큰 가중치를 두어 s_t update
- AdaGrad의 학습률 소멸 문제 해결

Various Optimizers – Adam

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t)$
 - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$
 - $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
 - $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
 - $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$
-
- Momentum과 RMSProp의 장점을 합침
 - Deep learning에서 가장 범용적으로 쓰임

Thank you